

Project assignment

Testing

Version 1.0.0 - May 2025

© Maurizio Morisio, Luca Ardito, Tommaso Fulcini, Stefano Mancini, 2025

Contents

- ORM Testing
- Multi-layer Testing
- Jest

ORM Testing

ORM Testing

- Tests must not touch the real application database to avoid altering persistent data
- First approach is mocking lower persistence layer: allows fast, isolated tests without touching the database
- In some cases (e.g. end-to-end tests), mocking is not possible because the implementation is unknown or external
 - In those cases, use a separate test database, ideally an in-memory SQLite DataSource
- **Never use the application's real database during testing**
- Always ensure tests run on an isolated and disposable DataSource

ORM Testing

- Entities are pure data holders, they contain no logic and do not need to be tested directly
- Logic is located in repositories and services, and these are the primary targets for testing
- When using an ORM, avoid testing the ORM itself—focus on your application logic
- Two main strategies:
 - Mocking the repository (unit testing)
 - Test DataSource (realistic but heavier)

Entity – Testing

- Entities are simple data holders, representing the structure of the database
- They do not contain business logic, only fields, relationships, and metadata (e.g., constraints)
- Testing them is not necessary, as their correctness is enforced by
 - The ORM itself, which ensures mappings are correctly applied
 - The database schema, which enforces constraints (e.g., foreign keys, uniqueness)
 - Other application tests, where entities are implicitly used and validated

Repository – Testing

- A repository should be tested at different levels while ensuring that database interactions are always mocked.
- Unit Testing the Repository
 - Standard ORM operations (save(), findById(), delete()) should be mocked, ensuring the repository correctly interacts with the ORM.
- Unit Testing Higher-Level Components
 - Repository itself, if involved, should be mocked entirely.
- Integration Testing the Repository
 - The repository itself should not be mocked
 - All its internal ORM operations (save, delete, find) must be mocked.

Mocking the Persistence Layer

- The recommended approach for unit testing business logic
- Replace TypeORM's native repository methods with Jest mocks
 - Example: mock `find()`, `findOne()`, `save()`, `remove()`, `delete()` methods
- Useful for isolating code and testing logic without persistence
- Avoids the need to configure or initialize any real database

Test DataSource

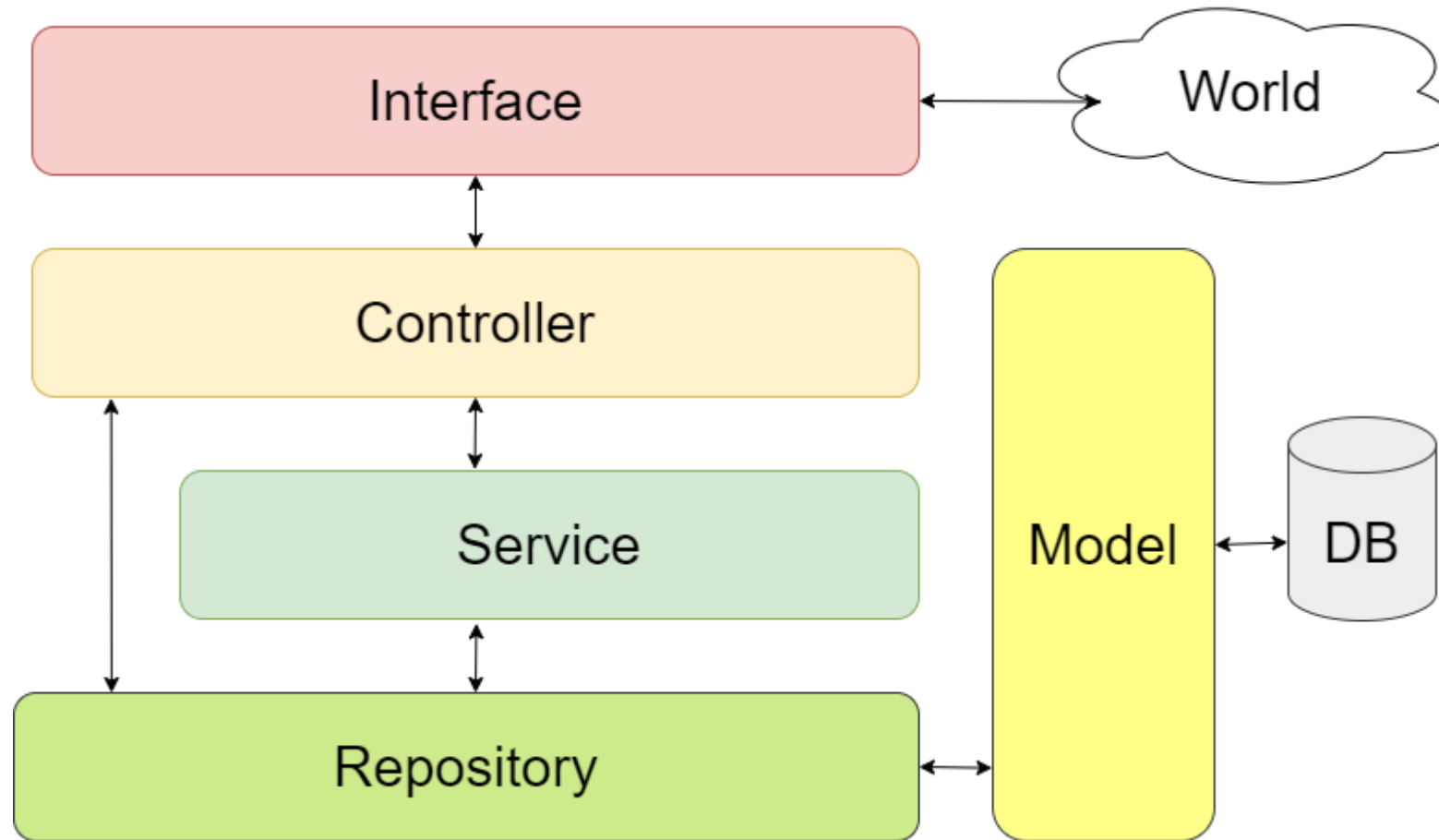
- Suitable for end-to-end tests
- Use a real database: SQLite in memory
- Database is recreated from scratch on each run
- Closer to real behavior, but slower than mocks
- Requires full DataSource management: initialization, cleanup and release

SQLite

- Lightweight, file-based relational database that does not require a separate server
- Used for embedded applications, prototyping, and **testing**
- Fully supported by TypeORM for file-based and **in-memory databases**
- SQLite Operating Modes
 - File-based: stores data in a .sqlite file, persisting between sessions
 - `database: "database.sqlite"`
 - **In-memory: runs entirely in RAM, losing all data when the app stops**
 - `database: ":memory:"`
 - Useful for testing and temporary data storage

Multi-layer testing

ORM-based Layered Structure



Testing Interface (routes) layer

- The route layer should not contain business logic
 - Extracts data from HTTP requests (req.params, req.body, etc.)
 - Calls middleware (e.g., auth)
 - Invokes the correct controller method
- Unit testing is rarely useful here
 - Routes delegate everything to middleware and controllers
- Integration testing is recommended
 - Use tools like supertest to simulate HTTP calls
 - Verify that middleware is triggered
 - Check routing, authentication, and response codes
 - Ensure the full stack (route → controller → service → DB) works correctly
- Route logic is minimal, but must be tested via integration tests to ensure the application behaves correctly from the outside.

Testing Controller layer

- Coordinates service and repository calls
- Filters or transforms results if needed
- Does not throw errors directly
- Propagates errors from services or repositories, handled globally by route-level error middleware
- Unit Testing
 - Mock services and (eventually) repositories
 - Verify:
 - Call sequences and parameters
 - Correct propagation of results and errors
 - Flow control logic
- Integration Testing
 - Use real services
 - Verify controller behavior in a realistic setup
 - Ensure that results and errors propagate correctly through the real layers
- Controllers implement application flow orchestration, not domain or HTTP logic. They deserve both unit tests for logic and integration tests for realistic interactions.

Testing Service layer

- Contains the real application logic
 - Validating domain rules
 - Transforming and combining data
 - Throwing domain-specific errors
- Services are stateless and purely logical
 - Easy to mock dependencies
 - Easy to test in isolation
- Unit Testing
 - Mock repositories and other services
 - Verify:
 - Business rules are applied correctly
 - Correct method calls and data flow
 - Expected errors are thrown in edge cases
- Integration Testing
 - Useful for critical flows involving multiple dependencies
 - Verify correct interaction with the persistence layer
- The service layer should be thoroughly unit tested and integration tested for full reliability.

Testing Repository layer

- Encapsulates database access through an ORM
- Exposes CRUD and query operations
- May implement light custom logic
 - Throwing domain-specific errors (e.g., "not found", "conflict")
 - Applying application-level filtering rules
- Unit Testing
 - Test each custom method in isolation
 - Verify logic and error handling
 - Ensure correct delegation to ORM methods
- Integration Testing
 - Useful for testing query correctness or ORM-specific behavior
- Repositories with custom logic are not pure data mappers. They should be unit tested like any other business component, especially when they enforce rules or throw errors.

Testing Model layer

- Entities
 - Represent database tables (mapped via ORM)
 - Contain no business logic
 - Already validated by the ORM itself
- DTOs (Data Transfer Objects)
 - Auto-generated from the OpenAPI/Swagger spec
 - May include methods for serialization/deserialization
 - Auto-generated code should not be modified
- Custom Error Classes
 - Simple data holders (e.g., NotFoundError, ConflictError)
 - No custom logic beyond storing information
- Model layer contains either auto-generated code or pure data holders
- **Testing is unnecessary, since there is no custom logic and no manual modifications are expected**

Jest

Jest Framework

- Jest is a JavaScript testing framework developed and maintained by Meta (Facebook)
- Designed for unit, integration, and end-to-end testing
- Provides:
 - A simple and expressive syntax (test, expect, etc.)
 - Built-in mocking and spying
 - Code coverage reports
 - Parallel test execution for performance
- Commonly used with Node.js, TypeScript, and frontend frameworks (React, Vue)

Configuration

- All the configuration is in `jest.config.ts` file
- Core settings:
 - `preset: "ts-jest"`
 - transpile TypeScript
 - `testEnvironment: "node"`
 - use Node.js APIs (for back-end)
 - `roots: ["<rootDir>/test"]`
 - look for test files here
 - `transform`
 - compile `.ts/.tsx` via `ts-jest`
 - `moduleNameMapper`
 - supports path aliases from `tsconfig.json`

Configuration - Coverage

- Ensures test quality and completeness
- Detects dead or untested code
- Enable coverage collection:

- `collectCoverage: true`

- Select files to include/exclude:

```
collectCoverageFrom: [  
  "src/**/*.{ts,tsx}",  
  "!src/models/**",           // skip DTOs/entities  
  "!src/config/**",           // skip constants  
  "!src/database/connection.ts",  
  "!test/**"  
],  
coveragePathIgnorePatterns: [  
  "<rootDir>/src/models/",  
  "<rootDir>/src/services/loggingService.ts",  
  "<rootDir>/test/"  
]  
]
```

Execution

- In package.json scripts

```
"test": "jest --coverage"
```
- Can be run via

```
npm test
```
- This runs all tests (in /test folder) and generates a coverage report
- To run a specific test file in the console use the following command

```
npm test UserRepository.mock.test.ts
```
- Report output:
 - Shown in terminal (text summary)
 - Full report in `coverage/lcov-report/index.html`

Report – Console

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	72.26	39.39	40.32	72.57	
src	72	22.22	22.22	73.46	
app.ts	100	100	100	100	
utils.ts	48.14	22.22	22.22	50	28-31,35-51
src/controllers	62.5	0	40	62.5	
authController.ts	50	0	0	50	9-14
UserController.ts	71.42	100	50	71.42	16-17,21-22
src/middlewares	100	0	100	100	
authMiddleware.ts	100	0	100	100	10
errorMiddleware.ts	100	100	100	100	
src/repositories	100	100	100	100	
UserRepository.ts	100	100	100	100	
src/routes	61.05	100	3.7	61.05	
authenticationRoutes.ts	66.66	100	0	66.66	9-12
gatewayRoutes.ts	64.28	100	0	64.28	8,13,18,23,28
measurementRoutes.ts	63.15	100	0	63.15	11,19,25,32,40,48,56
networkRoutes.ts	64.28	100	0	64.28	8,13,18,23,28
sensorRoutes.ts	64.28	100	0	64.28	8,13,18,23,28
userRoutes.ts	52	100	25	52	19,25-29,38-41,51-55
src/services	86.66	50	90.9	86.66	
authService.ts	80	40	100	80	25,31,39,47,52
errorService.ts	100	50	100	100	9-15,19
mapperService.ts	90	75	83.33	90	20

Test Suites: 5 passed, 5 total
Tests: 13 passed, 13 total
Snapshots: 0 total
Time: 21.626 s, estimated 23 s
Ran all test suites.

Report – Web Page

coverage/lcov-report/index.html

All files

72.26% Statements 172/238 39.39% Branches 13/33 40.32% Functions 25/62 72.57% Lines 172/237

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ▲		Statements ▾		Branches ▾		Functions ▾		Lines ▾	
src		72%	36/50	22.22%	2/9	22.22%	2/9	73.46%	36/49
src/controllers		62.5%	15/24	0%	0/1	40%	2/5	62.5%	15/24
src/middlewares		100%	12/12	0%	0/1	100%	3/3	100%	12/12
src/repositories		100%	12/12	100%	0/0	100%	7/7	100%	12/12
src/routes		61.05%	58/95	100%	0/0	3.7%	1/27	61.05%	58/95
src/services		86.66%	39/45	50%	11/22	90.9%	10/11	86.66%	39/45

Creating a Test DataSource

- It is necessary to use a dedicated test database without modifying the application code
 - Test database is isolated, fast, and auto-cleaned
- The app uses a shared `AppDataSource` object for DB access
- In tests, we create a new `TestDataSource` (e.g., SQLite in-memory)
- We override `AppDataSource` using
`Object.assign(AppDataSource, TestDataSource);`
 - `Object.assign(...)` copies all properties from `TestDataSource` to `AppDataSource`
- All modules importing `AppDataSource` will transparently use the test DB

Creating a Test DataSource

- AppDataSource

- The application creates and exports its DataSource

```
export const AppDataSource = new DataSource({
  type: CONFIG.DB_TYPE as any,
  database: CONFIG.DB_NAME,
  entities: CONFIG.DB_ENTITIES,
  synchronize: true,
  logging: false,
  host: CONFIG.DB_HOST,
  port: CONFIG.DB_PORT,
  username: CONFIG.DB_USERNAME,
  password: CONFIG.DB_PASSWORD
});
```

- TestDataSource

- In the test context, a dedicated DataSource is created and it replaces the one created by the application

```
export const TestDataSource = new DataSource({
  type: "sqlite",
  database: ":memory:",
  synchronize: true,
  entities: CONFIG.DB_ENTITIES
});

export async function initializeTestDataSource(): Promise<void> {
  if (!TestDataSource.isInitialized) {
    await TestDataSource.initialize();
    Object.assign(AppDataSource, TestDataSource);
  }
}
```

Implementing tests

- Test suite
 - Defined with `describe(name, fn)`
 - Groups related tests under a common name
 - Useful for organizing tests by module or feature
- Test case
 - Defined with `test(name, fn)` or `it(name, fn)`
 - Represents a single unit of behavior to verify
 - Both `test` and `it` are aliases, they behave the same
- A `describe` block can contain multiple `test/it` blocks
- Each test contains one or more `expect()` assertions
- Create one file per test suite, it keeps the project clean, modular, and easy to navigate

Test execution options

- `only(name, fn)`
 - Runs only the marked test or describe block
 - Works with `test`, `it`, and `describe`
- `skip(name, fn)`
 - Skips the marked test or describe block
 - Works with `test`, `it`, and `describe`
- `todo(name)`
 - Marks a placeholder for a future test
 - Only works with `test` or `it`
 - Fails if you provide a function
- `failing(name, fn)`
 - Marks a test expected to fail (passes if it throws)
 - Works with `test` and `it`
- If no `.only()` is used, all tests run normally

```
test.only("test to execute", () =>{})
```

```
test.skip("test to skip", () =>{})
```

```
test.todo("test to complete")
```

```
test.failing("test that fails", () =>{})
```

Implementing tests - Mocking

- Isolate logic under test
- Prevent real side effects (e.g., DB writes, network calls)
- Simulate error conditions and edge cases
- `jest.fn()`
 - Creates a standalone mock function
 - Tracks calls, arguments, and return values
 - Can be used to simulate callbacks or injected functions
- `jest.spyOn()`
 - Intercepts a method of an existing object
 - Allows tracking and optional overriding of the original implementation
 - Useful for verifying behavior without fully mocking a module
- `jest.mock()`
 - Replaces an entire module with mocked exports
 - Used to isolate the unit under test from external dependencies
 - Automatically applies mocks for all imported functions in the module

Jest - Mocking

- ***jest.fn()*** defines a new, undefined mock function which can be assigned to other functions and variables
- additional functions can be attached to it to define the behavior of the mock function
- **mockReturnThis()** returns the *this* object
- **mockReturnValue(value)** returns *value*
- **mockResolvedValue(value)** resolves *value* (for asynchronous functions)
- **mockRejectedValue(value)** rejects *value* (for async functions)
- **mockImplementation(val => 1 + val)** substitutes the implementation with the one in the brackets
- **mock_Once(value)** returns/resolves/rejects **value** the first time the mocked function is called, other calls must have a default mock call

Implementing tests - Mocking

```
const mockFind = jest.fn();
const mockSave = jest.fn();
const mockRemove = jest.fn();

jest.mock("@database", () => ({
  AppDataSource: {
    getRepository: () => ({
      find: mockFind,
      save: mockSave,
      remove: mockRemove
    })
  }
}));
```

- Creates mock functions to replace the standard repository `find`, `save`, `remove` methods
- Replaces the `@database` module with a custom mock
- Any code that imports `AppDataSource.getRepository(...)` will now receive mocked methods instead of the real ones

Implementing tests - Mocking

```
export class UserRepository {
  import UserDao
  private repo: Repository<UserDao>;

  constructor() {
    this.repo = AppDataSource.getRepository(UserDao);
  }

  async createUser(
    username: string,
    password: string,
    userType: UserType
  ): Promise<UserDao> {
    throwConflictIfFound(
      await this.repo.find({ where: { username } }),
      () => true,
      `User with username '${username}' already exists`
    );

    return this.repo.save({
      username: username,
      password: password,
      type: userType
    });
  }
}
```

- The custom repository internally builds a standard TypeORM repository via `AppDataSource.getRepository(...)`
 - In the test environment, this module (`@database`) is mocked to inject custom behavior
- The function `createUser` uses the standard repository `find` and `save` methods
 - These invocations will be replaced by the mocked functions

Implementing tests - Mocking

```
it("create user", async () => {
  mockFind.mockResolvedValue([]);

  const savedUser = new UserDAO();
  savedUser.username = "john";
  savedUser.password = "pass123";
  savedUser.type = UserType.Admin;

  mockSave.mockResolvedValue(savedUser);

  const result = await repo.createUser("john", "pass123", UserType.Admin);

  expect(result).toBeInstanceOf(UserDAO);
  expect(result.username).toBe("john");
  expect(result.password).toBe("pass123");
  expect(result.type).toBe(UserType.Admin);
  expect(mockSave).toHaveBeenCalledWith({
    username: "john",
    password: "pass123",
    type: UserType.Admin
  });
});
```

- `mockFind.mockResolvedValue([])`
 - Makes the `mockFind` to return an empty array of user, so the creation can proceed
- `mockSave.mockResolvedValue(savedUser)`
 - Makes the `mockSave` return a fake saved user
- The test runs the real method `createUser`
 - this internally uses the mocked `find` and `save`
- `expect(mockSave).toHaveBeenCalledWith(...)`
 - Verifies that the repository passed the correct data to the DB layer
- The custom logic of the repository is preserved and tested, but the underlying data layer is fully controlled via mocks

Supertest

- Node.js library for testing HTTP servers
 - Simulates real HTTP requests against your Express app
 - Returns full HTTP responses: status, headers, body
 - Built-in methods: `.get()`, `.post()`, `.put()`, `.delete()`, etc.
 - Chainable assertions: `.expect(status)`, `.expect(body)`, etc.
 - Works seamlessly with Jest
- Verifies:
 - HTTP status codes
 - Response body and headers
 - Authentication and validation flows
- Does not require the server to be started (works with the app object directly)
- Used in e2e tests to test how the app responds to external requests
- Also useful for route-level integration tests (without testing full service logic)

Supertest

```
import request from "supertest";

it("get all users", async () => {
  const res = await request(app)
    .get("/api/v1/users")
    .set("Authorization", `Bearer ${token}`);

  expect(res.status).toBe(200);
  expect(res.body.length).toBe(3);

  const usernames = res.body.map((u: any) => u.username).sort();
  const types = res.body.map((u: any) => u.type).sort();

  expect(usernames).toEqual(["admin", "operator", "viewer"]);
  expect(types).toEqual(["admin", "operator", "viewer"]);
});
```

- With Supertest it is possible to send an HTTP request, customize it and inspect the HTTP response (status, body, etc.)
- `request(app).get(...)`
 - Sends a real HTTP request to the Express app (no server needed)
- `.set("Authorization", ...)`
 - Sets custom headers for the request
- `await`
 - Waits for the full HTTP response (status, body, etc.)
- `expect(res.status).toBe(...)`
 - Checks the returned HTTP status code
- `expect(res.body)`
 - Inspects the parsed JSON response

Testing Asynchronous Code

- Asynchronous operations must be properly awaited, or tests may pass falsely or behave inconsistently
 - Declare test functions as `async`
 - Use `await` for all asynchronous operations (queries, service calls, supertest, etc.)
 - Ensures Jest waits before evaluating assertions
- Use `.rejects.toThrow(...)` for asynchronous error cases
 - `await expect(asyncCall()).rejects.toThrow(SomeError);`
- Avoid `done()`
 - Callback-based testing (`done`) is error-prone and discouraged
 - `async/await` is cleaner and safer in modern codebases
- Always await promises inside tests, `beforeEach`, `beforeAll`, etc.
- Use `jest.clearAllMocks()` in `beforeEach` or `afterEach` to avoid side effects
- Use `expect.assertions(n)` to ensure all assertions are executed
- `async` tests should be handles like any `async` function: use `await`, avoid `done`, and always return or await what matters